ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА

ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №2

«Структури даних та функціональна декомпозиція»

з курсу «Професійний Python»
Варіант №1 — «Аналіз системи обліку студентів»

Виконала: Руссу Анастасія Сергіївна

Група: ФЕП-32

Перевірив: Куньо І. М.

Дата: 25.10.2026

Львів — 2026

1. Мета роботи

Навчитися обирати структури даних Python відповідно до задачі та розподіляти обробку інформації між невеликими функціями. Закріпити використання list, tuple, set, dict, модуля collections, comprehensions, функцій вищого порядку, замикань і декораторів. Порівняти теоретичну складність пошуку з результатами вимірювання часу.

2. Індивідуальний варіант і постановка задачі

Варіант №1 — «Аналіз системи обліку студентів». Для кожного студента задано ім'я, прізвище, групу та середній бал. Необхідно показати всі записи, визначити унікальні групи, загальний середній бал і найкращого студента; відібрати записи з балом вище заданого; сформувати групування, рейтинг, кількість студентів у кожній групі та словниковий індекс.

Програма продовжує тему обліку студентів із першої лабораторної роботи. Дані зберігаються у списку словників. Кожен запис додатково містить унікальний ID для пошуку. Пороговий бал та ID потрібного студента задаються під час запуску програми.

3. Структури даних та їх призначення

Структура	**Використання і причина вибору**
list	Набір студентів, результати відбору й рейтингу. Зберігає порядок записів.
tuple	STUDENT_FIELDS та кортежі рядків у data.py; незмінний порядок полів.
set	Унікальні назви груп без повторів.
dict	Поля одного студента та індекс ID → запис для швидкого пошуку.
defaultdict(list)	Накопичення студентів у списках відповідних груп.
Counter	Підрахунок кількості студентів у кожній групі.

Словниковий індекс будується за ID, оскільки прізвища можуть повторюватися. Наприклад, у наборі є Марія Дорош і Тарас Дорош. Повторний ID викликає ValueError. Для середнього порожнього набору повертається None, щоб відрізнити відсутність даних від нульового бала.

4. Структура проєкту та опис модулів

```
Lab_02_variant_1/
├── pyproject.toml
├── README.md
├── .gitignore
├── src/
│   └── data_processor/
│       ├── __init__.py
│       ├── data.py
│       ├── processors.py
│       ├── analytics.py
│       ├── decorators.py
│       ├── main.py
│       └── benchmark.py
└── tests/
```

```
└── test_processing.py
```

Код розподілено між модулями за призначенням: дані, обробка колекцій, обчислення, декоратор і виведення результатів. У benchmark.py порівнюється час пошуку у списку та словнику. Каталог tests містить автоматизовані тести.

4.1. Вхідний набір у data.py

```
STUDENT_FIELDS = ("id", "first_name", "last_name", "group", "average_grade")


def get_students() -> list[dict]:
    rows = [
        (1, "Марія", "Дорош", "ФЕП-32", 86.5),
        (2, "Данило", "Левицький", "ФЕП-31", 95.0),
        (3, "Соломія", "Климчук", "ФЕП-32", 91.5),
        (4, "Остап", "Мазур", "ФЕП-31", 78.0),
        (5, "Олена", "Романюк", "ФЕП-33", 85.0),
        (6, "Тарас", "Дорош", "ФЕП-33", 72.0),
    ]
    return [dict(zip(STUDENT_FIELDS, row)) for row in rows]
```

Функція get_students щоразу створює новий список словників. Кортеж STUDENT_FIELDS визначає порядок полів, а zip пов’язує назви з відповідними значеннями. Шість студентів належать до трьох груп. Дані можна змінити у цьому модулі без змін алгоритмів обробки.

4.2. Пошук, фільтрація та групування

Модуль processors.py відокремлює обробку колекцій від виведення. Універсальні filter_items і sort_items приймають функцію як аргумент. Це дозволяє змінювати умову відбору чи ключ сортування без переписування самих алгоритмів.

```
from collections import Counter, defaultdict
from collections.abc import Callable


def get_unique_groups(students: list[dict]) -> set[str]:
    return {student["group"] for student in students}


def create_student_index(students: list[dict]) -> dict[int, dict]:
    index = {student["id"]: student for student in students}
    if len(index) != len(students):
        raise ValueError("ID студентів мають бути унікальними.")
    return index


def find_linear(students: list[dict], student_id: int) -> dict | None:
    for student in students:
        if student["id"] == student_id:
            return student
    return None


def filter_items(items: list[dict], predicate: Callable) -> list[dict]:
    return [item for item in items if predicate(item)]
def sort_items(items: list[dict], key: Callable,
               reverse: bool = False) -> list[dict]:
    return sorted(items, key=key, reverse=reverse)
```

```
def group_students(students: list[dict]) -> dict[str, list[dict]]:
    grouped = defaultdict(list)
    for student in students:
        grouped[student["group"]].append(student)
    return dict(grouped)


def count_students_by_group(students: list[dict]) -> Counter:
    return Counter(student["group"] for student in students)
```

У get_unique_groups використано set comprehension, у create_student_index — dict comprehension, у filter_items — list comprehension. defaultdict створює порожній список для нової групи. Counter отримує генератор назв груп та рахує їх появи.

4.3. Агрегація, аргументи та замикання

```
from collections.abc import Callable
from data_processor.decorators import measure_time


@measure_time
def calculate_average(students: list[dict]) -> float | None:
    if not students:
        return None
    return sum(s["average_grade"] for s in students) / len(students)


def find_best_student(students: list[dict]) -> dict | None:
    return max(students, key=lambda s: s["average_grade"], default=None)


def calculate_average_values(*values: float) -> float | None:
    if not values:
        return None
    return sum(values) / len(values)


def create_record(**fields) -> dict:
    return dict(fields)


def create_grade_filter(threshold: float) -> Callable[[dict], bool]:
    def predicate(student: dict) -> bool:
        return student["average_grade"] > threshold

    return predicate
```

calculate_average обчислює суму балів генераторним виразом і ділить її на кількість студентів. max із lambda знаходить запис із найбільшим балом; при рівних значеннях повертається перший. Параметр default=None забезпечує роботу з порожнім списком.

calculate_average_values приймає довільну кількість позиційних аргументів через *values. create_record отримує іменовані аргументи через **fields і повертає словник. У main ці функції використано для середнього бала та підсумку набору.

```
grades = tuple(s["average_grade"] for s in students)
calculate_average_values(*grades)
create_record(count=len(students), average=average,
            groups=len(get_unique_groups(students)))
```

create_grade_filter повертає predicate. Внутрішня функція пам'ятає threshold після завершення зовнішньої функції: це замикання. Для умови «вище заданого» використано >, тому бал, рівний порогу, не проходить відбір. Створені фільтри з різними порогами працюють незалежно.

4.4. Декоратор вимірювання часу

```
from functools import wraps
from time import perf_counter


def measure_time(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = perf_counter()
        result = func(*args, **kwargs)
        elapsed = perf_counter() - start
        print(f"Час {func.__name__}: {elapsed:.8f} с")
        return result

    return wrapper
```

Декоратор measure_time обгортає calculate_average. Він запам'ятовує час до виклику, виконує функцію, виводить тривалість і повертає її результат. *args та **kwargs передають аргументи без зміни. functools.wraps зберігає ім'я та інші метадані вихідної функції.

4.5. Послідовність роботи main.py

main зчитує --threshold і --student-id через argparse, перевіряє діапазон порога та отримує дані. Далі послідовно виводить усі передбачені варіантом результати. print_students відповідає лише за таблицю та повідомлення про порожню вибірку. Основні виклики:

```
students = get_students()
average = calculate_average(students)
best = find_best_student(students)
selected = filter_items(students, create_grade_filter(args.threshold))
ranking = sort_items(students,
                     key=lambda s: s["average_grade"], reverse=True)
grouped = group_students(students)
counts = count_students_by_group(students)
index = create_student_index(students)
found = index.get(args.student_id)
```

Сортування повертає новий список. Параметр reverse=True задається іменовано, тоді як students передається позиційно. Результат пошуку відсутнього ID дорівнює None. Перевірка __name__ дозволяє імпортувати функції без запуску програми.

4.6. Додаткові можливості

Реалізовано універсальну фільтрацію за predicate, універсальне сортування за key, статистику через Counter, створення фільтрів замиканням і benchmark list/dict. Ці функції дозволяють змінювати умови обробки та отримувати статистику для різних наборів даних.

5. Конфігурація та встановлення

```
[build-system]
requires = ["setuptools>=70"]
build-backend = "setuptools.build_meta"

[project]
name = "student-data-processor"
version = "0.1.0"
description = "Student data analysis for laboratory work 2"
readme = "README.md"
requires-python = ">=3.11"
authors = [{name = "Руссу Анастасія"}]

[project.optional-dependencies]
test = ["pytest>=8"]

[project.scripts]
student-analysis = "data_processor.main:main"

[tool.setuptools.packages.find]
where = ["src"]

[tool.pytest.ini_options]
testpaths = ["tests"]
```

Для звичайного запуску достатньо стандартної бібліотеки Python. pytest встановлюється лише як додаткова залежність test. Пакет розміщено у src; точка входу student-analysis пов’язана з main. README містить команди запуску, формат даних та пояснення benchmark. У .gitignore виключено середовище, кеші й результати збирання.

6. Створення virtual environment та запуск

Для запуску проєкту використовується Python 3.11. У терміналі PowerShell із кореневого каталогу проєкту потрібно створити й активувати віртуальне середовище, встановити пакет та запустити програму:

```
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -e ".[test]"
python -m data_processor.main
python -m data_processor.main --threshold 90 --student-id 2
python -m data_processor.benchmark
python -m pytest -q
```

Параметр -e дозволяє використовувати зміни вихідного коду без повторного встановлення пакета. Позначення [test] додає pytest. Основна програма виводить результати аналізу, benchmark порівнює швидкість пошуку, а pytest запускає тести.

7. Результати роботи програми

Для наведеного набору сума середніх балів становить 508, а середній бал — 508 / 6 = 84,6667. Після округлення програма виводить 84,67. Найкращий результат має Данило Левицький — 95. Результат роботи програми:

```
PS C:\Users\SunValley2\Downloads\121 спец\Третій курс\П'ятий семестр\Професійний Python\Lab_02_variant_1> .\.venv\Scripts\python.exe -m data_processor.main
Усі студенти
ID   Прізвище та ім'я         Група     Бал
--------------------------------------------------
1    Дорош Марія              ФЕП-32    86.50
2    Левицький Данило         ФЕП-31    95.00
3    Климчук Соломія          ФЕП-32    91.50
4    Мазур Остап              ФЕП-31    78.00
5    Романюк Олена            ФЕП-33    85.00
6    Дорош Тарас              ФЕП-33    72.00

Унікальні групи: ФЕП-31, ФЕП-32, ФЕП-33
Час calculate_average: 0.00000830 с
Середній бал: 84.67

Найкращий студент
ID   Прізвище та ім'я         Група     Бал
--------------------------------------------------
2    Левицький Данило         ФЕП-31    95.00

Бал вище 85
ID   Прізвище та ім'я         Група     Бал
--------------------------------------------------
1    Дорош Марія              ФЕП-32    86.50
2    Левицький Данило         ФЕП-31    95.00
3    Климчук Соломія          ФЕП-32    91.50

Рейтинг студентів
ID   Прізвище та ім'я         Група     Бал
--------------------------------------------------
2    Левицький Данило         ФЕП-31    95.00
3    Климчук Соломія          ФЕП-32    91.50
1    Дорош Марія              ФЕП-32    86.50
5    Романюк Олена            ФЕП-33    85.00
4    Мазур Остап              ФЕП-31    78.00
6    Дорош Тарас              ФЕП-33    72.00
```

Рис. 1. Перегляд, аналіз, фільтрація та сортування даних студентів

Вибірка «вище 85» містить три записи. Олена Романюк із балом 85 не входить до неї, що підтверджує строгість умови. Рейтинг упорядковано за спаданням бала; початковий список зберігає порядок.

7.1. Групування та пошук

```
PS C:\Users\SunValley2\Downloads\121 спец\Третій курс\П'ятий семестр\Професійний Python\Lab_02_variant_1> .\.venv\Scripts\python.exe -m data_processor.main

Група ФЕП-32
ID   Прізвище та ім'я         Група     Бал
--------------------------------------------------
1    Дорош Марія              ФЕП-32    86.50
3    Климчук Соломія          ФЕП-32    91.50

Група ФЕП-31
ID   Прізвище та ім'я         Група     Бал
--------------------------------------------------
2    Левицький Данило         ФЕП-31    95.00
4    Мазур Остап              ФЕП-31    78.00

Група ФЕП-33
ID   Прізвище та ім'я         Група     Бал
--------------------------------------------------
5    Романюк Олена            ФЕП-33    85.00
6    Дорош Тарас              ФЕП-33    72.00

Кількість студентів у групах:
ФЕП-32: 2
ФЕП-31: 2
ФЕП-33: 2

Пошук ID 3
ID   Прізвище та ім'я         Група     Бал
--------------------------------------------------
3    Климчук Соломія          ФЕП-32    91.50
```



Рис. 2. Групування студентів, підрахунок їх кількості у групах і пошук за ID

Кожна з трьох груп містить по два студенти. Пошук ID 3 повертає Соломію Климчук. Середнє, отримане через *args, збігається з результатом агрегування списку; різниця у записі пояснюється округленням під час виведення.

8. Аналіз часової складності

Операція	Складність	Пояснення
Пошук ID у list	O(n)	У найгіршому випадку перевіряються всі записи.
Побудова dict-index	O(n), середня	Один прохід; додаткова пам’ять O(n).
Пошук ID у dict	O(1), середня	Хешування ключа; найгірший випадок O(n).
Унікальні групи	O(n), середня	Додавання груп до set; пам’ять O(g).
Фільтрація	O(n)	Одна перевірка умови для кожного запису.
Групування та Counter	O(n), середня	Один прохід із доступом до словника.
Рейтинг	O(n log n)	Найгірша складність sorted.
Середній бал і максимум	O(n)	Необхідно переглянути всі бали.

Тут n — кількість студентів, g — кількість груп. Оцінки припускають сталу вартість порівняння балів і роботи з короткими ключами. Big-O описує залежність від обсягу даних, а не точну кількість секунд.

9. Експериментальне порівняння list і dict

Мета експерименту — порівняти швидкість пошуку одного запису у списку та словниковому індексі при збільшенні кількості студентів.

Для 1 000, 10 000 та 100 000 студентів згенеровано записи з унікальними ID. Пошук виконується для останнього ID: list проходить усі записи, а dict використовує ключ. Перед вимірюванням перевірено, що обидва способи повертають той самий запис.

Для кожного набору проведено п’ять серій вимірювань. В одній серії пошук у list повторюється 100 разів, у dict — 100 000 разів. Медіану тривалості серій поділено на кількість повторів, щоб отримати час одного пошуку. Час побудови індексу наведено окремо.



Рис. 3. Порівняння часу пошуку студента у списку та словнику

Зі збільшенням кількості записів час пошуку у списку зростає. Для dict час одного пошуку майже не змінюється. Це відповідає теоретичним оцінкам O(n) для list і O(1) у середньому для dict. При повторному запуску числові результати можуть відрізнятися.

Словниковий індекс доцільно використовувати, коли пошук виконується багато разів. Для одного запиту потрібно також врахувати час його побудови та додаткову пам'ять. Таблиця результатів виводиться в терміналі.

10. Перевірка коректності

Виконано сім автоматизованих тестів. Перевірено середній бал і групування, строгий поріг та незалежність замикань, пошук і повторні ID, сортування без зміни вхідного списку, порожні набори, *args, **kwargs, збереження метаданих декоратором і незалежність повторно створених наборів.

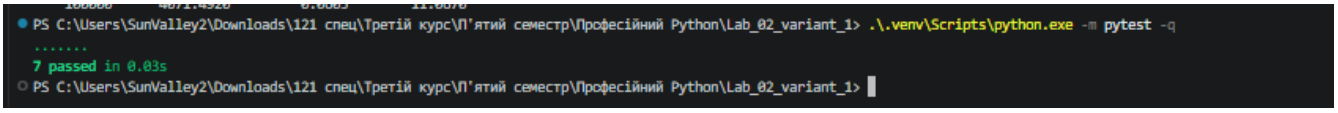


Рис. 4. Результат виконання автоматизованих тестів

Для порожнього набору середнє й найкращий студент дорівнюють None. При відсутньому ID таблиця містить повідомлення «Записів немає». Повторний ID відхиляється, а повторні прізвища підтримуються.

11. Відповіді на контрольні запитання

1. Що таке list?

Впорядкована змінювана колекція, яка дозволяє повтори й доступ за індексом.

2. Чим list відрізняється від tuple?

Список можна змінювати після створення; склад і порядок елементів кортежу змінити не можна.

3. Чому tuple називається immutable?

Не можна замінити, додати чи вилучити його елементи. Вкладений змінюваний об'єкт при цьому може змінювати власний стан.

4. Для чого використовується set?

Для унікальних значень, перевірки належності та операцій об'єднання, перетину й різниці.

5. Яка властивість елементів set є важливою?

Елементи мають бути хешованими: мати незмінний протягом життя хеш і узгоджену рівність.

6. Що таке dict?

Змінювана колекція пар ключ–значення з унікальними хешованими ключами.

7. Що таке key-value structure?

Структура, у якій за ключем отримують пов’язане значення, наприклад ID → запис студента.

8. Чому пошук key у dict зазвичай швидкий?

Хеш-таблиця дозволяє визначити місце ключа без послідовного перегляду всіх елементів.

9. Що таке hashability?

Здатність об’єкта мати сталий хеш. Рівні об’єкти повинні мати однаковий хеш.

10. Чому list не може бути key у dict?

Список змінюваний і не підтримує хешування, потрібне ключам словника.

11. Для чого використовується Counter?

Для підрахунку кількості поява значень, у роботі — студентів кожної групи.

12. Для чого використовується defaultdict?

Для автоматичного створення значення відсутнього ключа через задану фабрику, наприклад list.

13. Для чого використовується deque?

Для черги з ефективним додаванням і вилученням з обох кінців.

14. Що таке list comprehension?

Вираз побудови списку з ітерації та, за потреби, умови відбору.

15. Що таке set comprehension?

Вираз побудови множини; повтори результатів автоматично прибираються.

16. Що таке dict comprehension?

Вираз, що формує пари ключ–значення під час обходу колекції.

17. Що таке generator expression?

Вираз, який ліниво створює значення під час перебору без побудови всього проміжного списку.

18. Що означає O(1)?

Верхня асимптотична межа витрат не зростає зі збільшенням кількості елементів.

19. Що означає O(n)?

Витрати обмежені лінійною функцією розміру вхідних даних.

20. Що означає O(n log n)?

Витрати зростають пропорційно добутку n і логарифма n; приклад — найгірший час sorted.

21. Що означає O(n²)?

Квадратичне зростання витрат, наприклад перевірка всіх пар елементів.

22. Яка приблизна складність пошуку у list?

O(n) у найгіршому випадку, O(1), якщо потрібний запис стоїть першим.

23. Яка середня складність пошуку у set?

O(1) за звичайних умов хешування; найгірший випадок O(n).

24. Яка середня складність пошуку key у dict?

O(1), хоча при несприятливих колізіях можлива O(n).

25. Чому правильна структура даних може прискорити програму?

Вона скорочує кількість потрібних дій: індекс замінює повний прохід списку доступом за ключем.

26. Що таке функціональна декомпозиція?

Поділ задачі на функції, кожна з яких виконує одну логічну операцію.

27. Які переваги малих functions?

Їх легше читати, перевіряти окремо, повторно використовувати та змінювати.

28. Що таке positional argument?

Аргумент, який зіставляється з параметром за позицією у виклику.

29. Що таке keyword argument?

Аргумент, переданий із назвою параметра, наприклад reverse=True.

30. Для чого використовується *args?

Для приймання додаткових позиційних аргументів у кортеж. У виклику * розпаковує послідовність.

31. Для чого використовується **kwargs?

Для приймання додаткових іменованих аргументів у словник. У виклику ** розпаковує відповідне відображення.

32. Що таке lambda function?

Коротка анонімна функція, тіло якої складається з одного виразу.

33. Де доцільно використовувати lambda?

Для простого ключа sorted або max чи короткого предиката відбору.

34. Чому складну business logic не варто записувати в lambda?

Складний вираз важче читати й перевіряти; окрема іменована функція чіткіше пояснює дію.

35. Що таке closure?

Функція, яка зберігає доступ до змінних зовнішньої функції після завершення її виклику.

36. Які variables може пам’ятати closure?

Вільні змінні охоплюючої області, на які посилається її тіло; у роботі це threshold.

37. Що таке decorator?

Об’єкт, який отримує функцію або клас і повертає заміну, часто обгортку з додатковою поведінкою.

38. Для чого застосовуються decorators?

Для вимірювання часу, журналювання, перевірки доступу, кешування та інших повторюваних дій.

39. Для чого використовується functools.wraps?

Для перенесення метаданих обгорнутої функції та встановлення __wrapped__, що полегшує перевірку й документацію.

40. Чому потрібно оцінювати складність алгоритму?

Щоб передбачати поведінку при зростанні даних і обирати алгоритм з прийнятними витратами часу та пам’яті.

12. Висновок

У ході виконання лабораторної роботи розроблено модуль аналізу системи обліку студентів за варіантом №1. Для зберігання та обробки даних використано list, tuple, set, dict, defaultdict і Counter. Реалізовано пошук, відбір, групування, рейтинг та обчислення середнього бала, а також функції з *args і **kwargs, замикання й декоратор. Для шести записів отримано середній бал 84,67; найбільший бал становить 95. Сім тестів завершилися успішно. Експеримент на трьох розмірах наборів показав зростання часу лінійного пошуку та приблизно сталий час пошуку у готовому словниковому індексі. Вибір структури даних впливає на швидкість повторних операцій, а поділ на функції полегшує їх перевірку.